

---



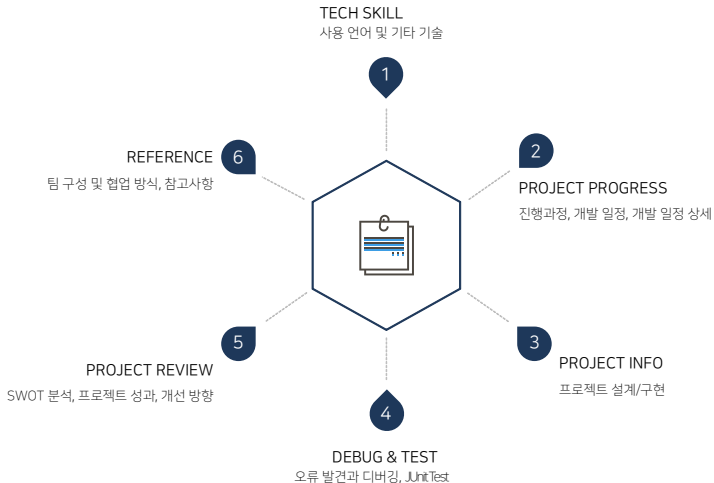
WEB PROJECT

Personal GYM

01\_Portfolio

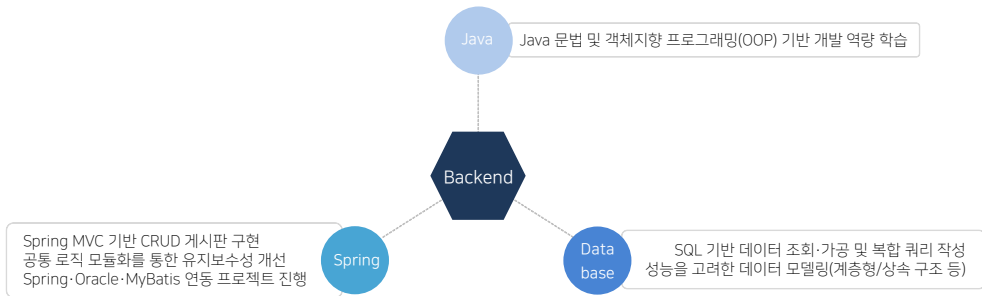
---

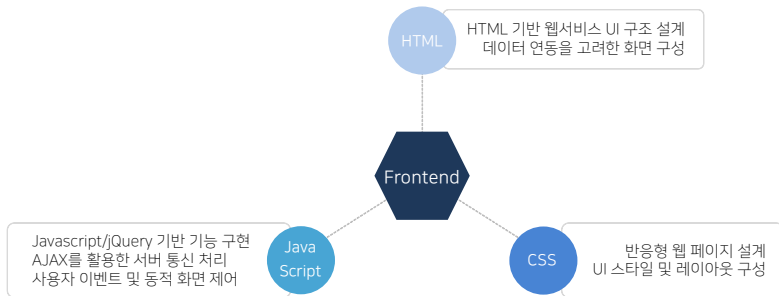
# MAIN INDEX



# 1 TECH SKILL

## [ 1-1 ] 사용 언어 및 기타 기술

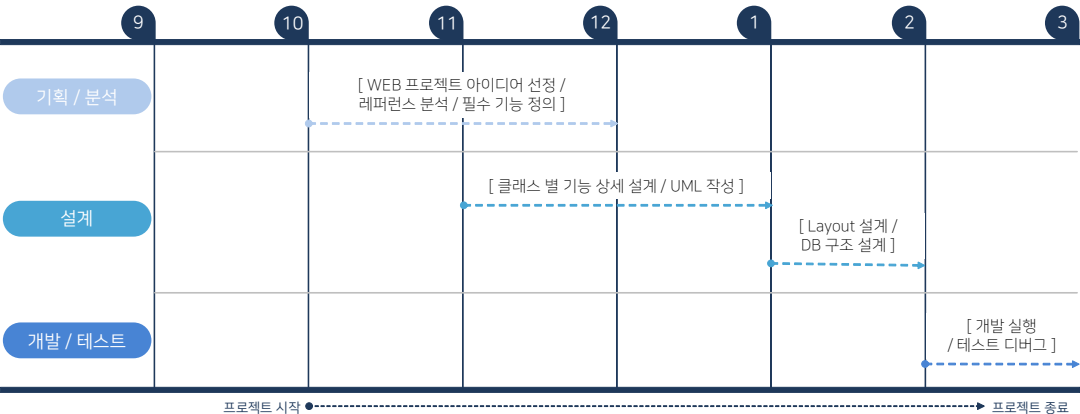




## 2

## PROJECT PROGRESS

## [ 2-1 ] 진행과정

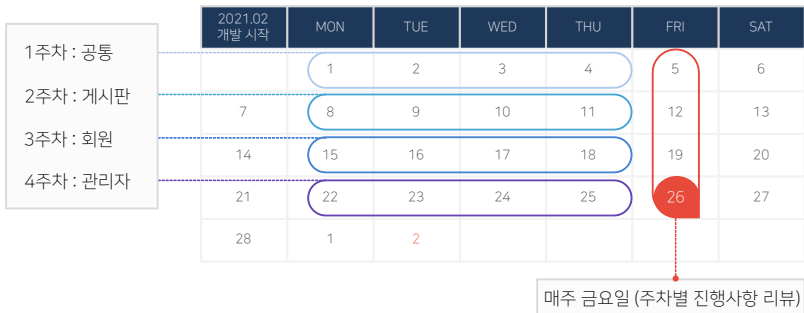


Gantt chart를 활용하여, 팀원과 일정 공유

## 2

## PROJECT PROGRESS

## [ 2-2 ] 개발 일정



진행 방식 : 애자일(Agile)기법 적용, 주차별 단위 개발 및 리뷰 진행

개발 시작

개발 종료

1주차

공통

 개발 내용

- Html,CssRefactoring  
(So-admin Template 참조)
- Header, Topbar, Sidebar 구현
- 게시판연결 및 홈 이동 기능 구현
- TABLE 구조 개선  
(<th> → caption 활용)
- 등록일/수정일 기능 개발

 핵심 기능

- 반응형 레이아웃 적용
- 공통 UI 모듈화
- 메뉴 및 네비게이션 구성
- 날짜 관리 표준화

2주차

게시판

 개발 내용

- 게시물, 댓글, 회원정보 DB 구조 설계
- 게시물 조회수 증가, 좋아요/싫어요 증감 (토글)
- 페이지징 처리 구현 (구글 페이지징 기법 참조)
- 해시태그 기능 구현
  - Komoran 형태소 분석 엔진 적용
  - 해시태그 생성 및 게시물 검색 처리
- 파일 유형별 업로드 기능 구현 및 제어

 핵심 기능

- 게시물, 댓글, 대댓글 CRUD 구현
- 게시물 관련 세부 기능 개발
- 해시태그 자동 추출 및 검색
- 파일 첨부 기능

3주차

회원

 개발 내용

- 회원 및 권한 테이블 DB 구조 설계
- 회원 CRUD 구현
- 로그인 / 로그아웃 구현
- 비밀번호 암호화 적용
- 임시 비밀번호 발급 및 Gmail 전송
- 로그인 상태별 Header UI 구현
- Logout Model 창 구현

 핵심 기능

- 회원 가입, 내 정보 보기/수정, 탈퇴
- 자동 로그인 적용
- 비밀번호 관련 세부 기능 개발
- 조건부 UI 제공

4주차

관리자

 개발 내용

- 회원 권한별 접근 제어 설정
- 관리자 권한 전용 기능 구현
  1. 회원 전체 목록 조회
  2. 회원 강제 탈퇴 (체크박스)
  3. 공지사항 CRUD 기능

 핵심 기능

- 사용자 권한 관리
- 회원 정보 관리
- 공지사항 운영

## PROJECT INFO INDEX

---

|       |                       |     |
|-------|-----------------------|-----|
| ■ 설 계 | UML 'UseCase Diagram' | 3-1 |
|       | UML 'Class Diagram'   | 3-2 |
|       | Web_CRUD 및 기타 기능      | 3-3 |
| ■ 구 현 | 처리 성능 향상              | 3-4 |
|       | 안정적인 유지 보수            | 3-5 |
|       | 효율적인 데이터관리            | 3-6 |
|       | 코드 품질 개선              | 3-7 |
|       | 기술적 문제 해결             | 3-8 |

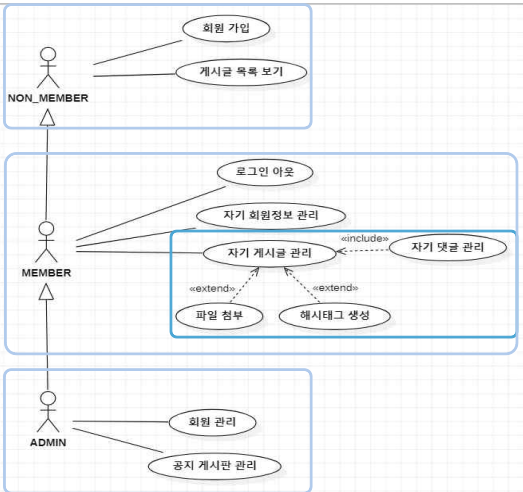
---





### 3 PROJECT INFO

#### 설 계 [ 3-1 ] UML 'UseCase Diagram'



#### 설계 특징

##### 1 사용자 권한 구조

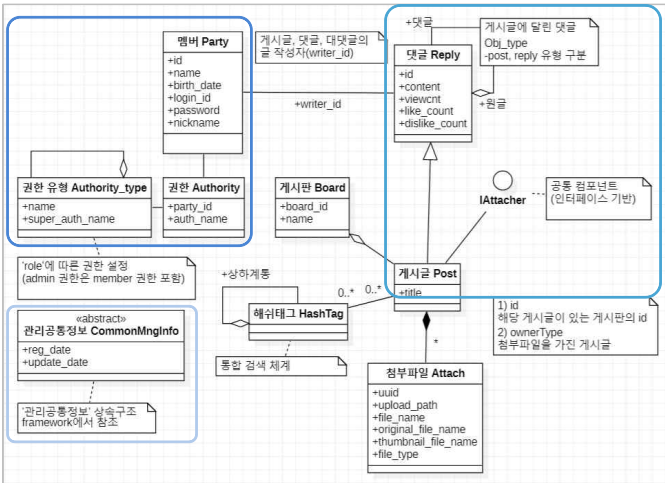
- 권한 : NON\_MEMBER / MEMBER / ADMIN
- 권한별 기능 접근 제어

##### 2 기능 관계 설계

- include 관계를 활용 : 필수 기능 의존 관계 표현
- 반드시 게시글이 있어야 댓글 및 대댓글 작성 가능
- extend 관계를 활용 : 선택 기능 분리 관계 표현
- 게시글 작성시 선택적 수행(파일첨부, 해시태그 생성)

### 3 PROJECT INFO

#### 설 계 [ 3-2 ] UML 'Class Diagram'



#### 설계 특징

- 1 공통 정보 상속
  - CommonMngInfo 기반
  - 등록일, 수정일 공통 관리
- 2 게시글, 댓글 모델 통합 설계
  - obj\_type 기반 객체 구분 (게시글, 댓글)
  - 댓글, 대댓글을 Reply 모델로 통합 관리
- 3 권한 기반 사용자 모델
  - Party / Authority 분리
  - Spring Security 연동

### 3 PROJECT INFO

설 계 [ 3-3 ] Web\_CRUD 및 기타 기능

|                          | Create                                         | Read                                            | Update                        | Delete         | 기타 기능                                                                                       |
|--------------------------|------------------------------------------------|-------------------------------------------------|-------------------------------|----------------|---------------------------------------------------------------------------------------------|
| Member (회원)              | 가입                                             | 내 정보 보기                                         | 내 정보 변경<br>+비밀번호 변경           | 탈퇴             | 일반 게시판 내 게시글 관리<br>임시 비밀번호 발급을 위한 이메일전송                                                     |
| Admin (관리자)              | <div>← Admin = Member 기능 포함 및 회원 관리 수행 →</div> |                                                 |                               |                | 공지사항 게시판 내 게시글 관리                                                                           |
|                          |                                                | 모든 회원 정보 열람                                     |                               | Member<br>강제탈퇴 |                                                                                             |
| Post (게시글)<br>Reply (댓글) | 작성                                             | 보기<br>- 목록 조회 (Table + Paging)                  | 수정                            | 삭제             | 1) 게시글 : 조회수 증가, 좋아요/싫어요<br>2) obj_type 기반 계층형 댓글/대댓글 구조                                    |
| Hashtag (해시태그)           | 생성                                             | 추출된 해시태그를<br>Post(게시글)에서 보기                     | 기존 첨부파일 삭제 및<br>신규 파일 재업로드 처리 | 삭제             | 1) 선택적으로 추가 가능<br>2) 생성된 해시태그로 게시글 검색 가능<br>(게시글과 N:M 관계로 관리)                               |
| Attach (첨부파일)            | 첨부 파일<br>업로드                                   | 게시글 - '첨부된 파일 목록' 보기<br>- 이미지 미리보기, 비디오 자동재생 지원 |                               |                | 1) 선택 첨부 가능<br>2) 오디오, 기타 형식 (다운로드 가능)<br>3) 실행 파일 및 압축 파일 업로드 제한<br>(exe, zip, alz, sh 형식) |

## 구현 [3-4] 처리 성능 향상

HashTagMapper.xml

```
<select id="selectNewID" resultType="Long">
  select max(newid)
  from (
    select 1 as newid from dual
    union
    select newid
    from (
      select (id + 1) as newid
      from T_Hash_Tag
      where rownum = 1
      order by id desc
    )
  )
</select>
<insert id="createHashTag">
  <foreach item="obj" collection="List"
    open="INSERT ALL "
    close=" SELECT * FROM DUAL">
    INTO T_Hash_Tag(id, word)
    VALUES ({obj.id}, #{obj.word})
  </foreach>
</insert>
```

## 기존 방식

- Sequence(nextval) 반복 호출
- DB 호출 횟수 : Insert 10회 실행



## 개선 방식

- Foreach 활용, 다중 데이터 일괄 삽입
- DB 호출 횟수 : 10회 → 2회

## 해시태그 등록 성능 최적화

- DB 호출 횟수 80% 감소
- 대량 데이터 처리 성능 향상

ReplyMapper.xml

```
<select id="findReplyById" resultMap="replyResultMap">
  <![CDATA[
    select p.board_id, p.id, p.content, p.writer_id, p.title, p.like_count,
    p.dislike_count, p.obj_type,
    p.reg_date, p.update_date,
    c.count_of_reply,
    w.id w_id, w.nickname w_nickname,
    ht.id ht_id, ht.word ht_word, ht.descript ht_descript
    from (
      select a.id, count(b.id) count_of_reply
      from T_Reply a
      left outer join T_Reply b on a.id = b.original_id
      where a.id = #{id}
      group by a.id
    ) c, T_Reply p
    left outer join T_Party w on p.writer_id = w.id
    left outer join M_HT_Reply mht on p.id = mht.reply_id
    left outer join T_Hash_Tag ht on mht.hash_tag_id = ht.id
    where c.id = p.id
  ]]>
</select>
```

## 기존 방식

- 불필요한 데이터 조회 발생



## 개선 방식

- where·join 활용 : 조회 조건 최적화
- SubQuery 활용 : 조회 대상 선별 추출

## 데이터 조회 쿼리 최적화

- 조회 효율 향상
- 응답 속도 개선

### 3 PROJECT INFO

#### 구 현 [ 3-5 ] 안정적인 유지 보수

changePW.jsp

```
$("#pw1").on("focusout", function isSame() {
    var pw1 = $("#pw1").val();
    // 정규 표현식: 숫자, 특수 각 1회 이상, 영문은 2개 이상 사용, 글자수 8~15제한
    var pwPattern = /^(?=.*\d{1,50})(?=.*[~!@#%&^&*()-+=]{1,50})(?=.*[a-zA-Z]{2,50}).{8,15}$/;

    if (isEmpty(pw1))
        return;
    else if (pw1.length < 8 || pw1.length > 15) {
        window.alert('The password can only be used with 8~15 characters. ');
        document.getElementById('pw1').value='';
        $("#pw1").focus();
    } else if (pwPattern.test(document.getElementById('pw1').value) != true){
        alert("Password doesn't fit the format");
        document.getElementById('pw1').value='';
        $("#pw1").focus();
    }
});

var frmChangePW = $("#frmChangePW");
$("#btnChangePW").on("click", function(e) {
    e.preventDefault();
    frmChangePW.submit();
});

// 넣어진 값이 빈값인지 체크합니다.
// !value 하면 생기는 논리 오류 제거는 명시적으로 value == 사용합니다.
// [], {} 도 빈값으로 처리합니다.
var isEmpty = function(value){
    if( value == "" || value == null || value == undefined ||
        ( value != null && typeof value == "object" && !Object.keys(value).length ) ){
        return true
    }else{
        return false
    }
};
```

HashTagVO.java

```
@Override
public boolean equals(Object obj) {
    if (this == obj)
        return true;
    if (obj == null)
        return false;
    if (getClass() != obj.getClass())
        return false;
    HashTagVO other = (HashTagVO) obj;
    if (word == null) {
        if (other.word != null)
            return false;
    } else if (!word.equals(other.word))
        return false;
    return true;
}
```

저장된 비밀번호 확인

시작 페이지 T.PARTY

로그인 Model | 계약 조건 | 권한 부여 | 통계 | 관리자 | 통계 | 내부 정보 | 분할 영역 | 검색 | SQL

ID NAME BIRTHDATE LOGNID PASSWORD

1 PADMIN 91/02/25 testportnas \$2a\$10\$\$.zvq0r0AGv1rg3Bz/kDQdevK1BC/LBFktCm2.6

2 5member 90/01/01 molpmember \$2a\$10\$Jif6ASoH3Zr6nxnDYJhdy9dQrT8B2G1YneOpQCad49ZeIYUti/F6

입력값 검증 및 예외 방지

- 정규식을 활용하여 비밀번호 형식 검증
- NullPointerException 발생 가능성 최소화
- 사용자 입력 오류를 사전에 검증 처리

# 3 PROJECT INFO

## ■ 구 현 [ 3-5 ] 안정적인 유지 보수

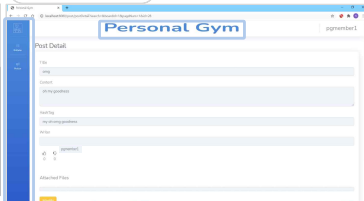
### postCommon.jsp

```
<script type="text/javascript">
//create read update
function setOperationMode(operationMode) {
    if ("read" === operationMode) {
        $("#title").attr("readonly", true);
        $("#txacontent").attr("readonly", true);
        $("#hashTag").attr("readonly", true);
        //btn: extractHashTag
        $("#btnExtractHashTag").hide();
        $("#clickTxt").hide();
        //btn: file select, file upload
        $("#uploadFilebl").hide();
        $("#uploadFile").hide();
        $("#btnUpload").hide();
        $("#clickAttachTxt").hide();
    } else if ("create" === operationMode) {
        $("#divIdentifier").hide();
        $("#btnLike").hide();
        $("#btnDislike").hide();
    } else if ("update" === operationMode) {
        $("#btnLike").hide();
        $("#btnDislike").hide();
    }
}
</script>
```

1. postDetail.jsp = read
2. registerPost.jsp = create
3. modifyPost.jsp = update

```
Case SENSITIVE Pattern ( ? = any character, * = any string)
setOperationMode
Matching Items:
L: Text
39 setOperationMode "create"
40 setOperationMode "view"
54 setOperationMode "read"
55 setOperationMode "update"
47 setOperationMode "change"
62 function setOperationMode(operationMode) {
54 setOperationMode "read"
47 setOperationMode "update"
59 function setOperationMode(operationMode) {
54 function setOperationMode(operationMode) {
```

### 게시글 보기 화면



### CommonMngInfoVO.java

```
package www.dream.com.framework.model;

import java.util.Date;

/**
 * 모든 테이블에 있는 관리 정보를 각 VO에 중복 개발하지 않고,
 * framework패키지 내부에 상속 구조를 활용하여 공통화합니다.
 */
public abstract class CommonMngInfoVO {
    @Getter @Setter
    @Caption(whenUse=WhenUse.all, caption="RegistrationDate")
    protected Date regDate;
    @Getter @Setter
    protected Date updateDate;
}
```

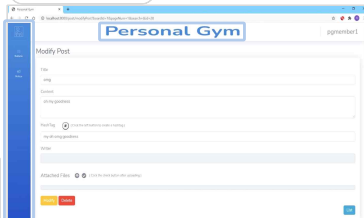
### PartyVO.java

```
package www.dream.com.party.model;

import java.io.Serializable;

@Data
@NoArgsConstructor
public class PartyVO extends CommonMngInfoVO implements Serializable
```

### 게시글 수정 화면



### 공통 모듈 구현 - 화면, 날짜

- Header, Footer, MenuBar 등
- 게시글 CRUD 기능별 UI 제어
- 날짜 통합 관리(등록일, 수정일)

# 3 PROJECT INFO

- 구 현 [3-5] 안정적인 유지 보수

## Criteria.java

```
package www.dream.com.framework.model;

@Getter
public class Criteria {
    /** 표시할 '페이지의 전체 개수' */
    private static final long PAGING_AMOUNT = 10;
    /** 한 페이지에서 보여줄 '게시글의 개수' */
    public static final int DEFAULT_AMOUNT = 10;

    public Criteria(long pageNum, long totalDataCount) {
        this.totalDataCount = totalDataCount;
        setPageNum(pageNum);
    }

    public long getOffset() {
        return (pageNum - 1) * DEFAULT_AMOUNT;
    }

    public long getLimit() {
        return pageNum * DEFAULT_AMOUNT;
    }

    public void setPageNum(long pageNum) {
        this.pageNum = pageNum;
        //아래는 구글 스타일을 활용합니다.
        endPage = pageNum + PAGING_AMOUNT / 2;
        endPage = endPage < PAGING_AMOUNT ? PAGING_AMOUNT : endPage;
        startPage = endPage - PAGING_AMOUNT + 1;
        int realEnd = (int) Math.ceil((float) totalDataCount / DEFAULT_AMOUNT);
        if (realEnd < endPage)
            endPage = realEnd;
        hasPrev = startPage > 1;
        hasNext = endPage < realEnd;
    }

    public void setTotalDataCount(long totalDataCount) {
        this.totalDataCount = totalDataCount;
        int realEnd = (int) Math.ceil((float) totalDataCount / DEFAULT_AMOUNT);
        if (realEnd < endPage)
            endPage = realEnd;
        hasNext = endPage < realEnd;
    }
}
```

## 게시판 페이징 화면

| Title               | Like | Dislike | View | Writer    | RegistrationDate |
|---------------------|------|---------|------|-----------|------------------|
| massive data create | 0    | 0       | 0    | ppmember1 | 2021-03-17       |
| massive data create | 0    | 0       | 0    | ppmember2 | 2021-03-17       |
| massive data create | 0    | 0       | 0    | ppmember2 | 2021-03-17       |
| massive data create | 0    | 0       | 0    | ppmember1 | 2021-03-17       |
| massive data create | 0    | 0       | 0    | ppmember2 | 2021-03-17       |
| massive data create | 0    | 0       | 0    | ppmember2 | 2021-03-17       |
| massive data create | 0    | 0       | 0    | ppmember1 | 2021-03-17       |
| massive data create | 0    | 0       | 0    | ppmember2 | 2021-03-17       |
| massive data create | 0    | 0       | 0    | ppmember2 | 2021-03-17       |
| massive data create | 0    | 0       | 0    | ppmember1 | 2021-03-17       |

## 회원목록 페이징 화면

| Check                    | ID | Login ID    | Name   | Nickname  | BirthDate  | RegistrationDate |
|--------------------------|----|-------------|--------|-----------|------------|------------------|
| <input type="checkbox"/> | 5  | no2ppmember | member | ppmember1 | 1990-01-05 | 2021-03-14       |
| <input type="checkbox"/> | 6  | no2ppmember | member | ppmember2 | 1993-03-17 | 2021-03-14       |
| <input type="checkbox"/> | 24 | member01    | member | member1   | 2021-03-15 | 2021-03-28       |
| <input type="checkbox"/> | 25 | member02    | test   | member2   | 2021-03-05 | 2021-03-28       |

## 공통 모듈 구현 - 페이징 처리

- Criteria 클래스로 페이징 로직 통합 관리  
→ 게시글, 댓글, 회원 목록에 활용
- 상수화, 규칙 정의로 유지보수 편의성 향상

### 3 PROJECT INFO

#### ■ 구 현 [ 3-6 ] 효율적인 데이터 관리

##### KOMORAN 한국어 형태소 분석 엔진의 분석 결과

| 형태소   | 품사  | 품사명   |
|-------|-----|-------|
| 대한민국  | NNP | 고유 명사 |
| 은     | JX  | 보조사   |
| 민주공화국 | NNP | 고유 명사 |

HashTagService

##### 저장된 해시태그 데이터

```
SELECT *  
FROM T_Hash_Tag  
START WITH word = 'IT'  
CONNECT BY PRIOR id = super_id;
```

| #ID  | SUPER_ID | WORD               |
|------|----------|--------------------|
| 1 39 | (null)   | IT                 |
| 2 40 | 39       | ProgramingLanguage |
| 3 41 | 40       | java               |
| 4 42 | 41       | thread             |
| 5 43 | 41       | lambda             |

DreamPair.java

##### 해시태그 처리 흐름

###### 사용자 입력

게시글, 댓글 등

###### 형태소 분석

품사 분석 및 키워드 추출

###### 해시태그 생성

추출된 키워드 단어집(Lexicon) 생성 및 인덱스 정렬

###### 중복 데이터 제거

기존 해시태그와 비교 후 중복 제거

###### 해시태그 저장

최종 해시태그 DB 저장

###### 계층형 해시태그 검색

상위-하위 관계 기반 검색 제공(CONNECT BY)

##### 주요 효과



###### 운영 관리 부담 감소

형태소 분석 기반 키워드 해시태그 자동추출



###### 데이터 정합성 향상

중복 데이터 제거로 데이터 품질 개선



###### 검색 정확도 및 편의성 향상

계층형 구조로 빠르고 정확한 검색 지원



## 구현 [3-7] 코드 품질 개선

ClassUtil.java

```
public class ClassUtil {
    public static List<Field> getListField(Class<?> targetClass, Class<Caption> annotation) {
        List<Field> listField = new ArrayList<>();
        //부수효과(Side Effect) 활용. 인자에 내용이 호출로 바뀝니다.
        getAllField(targetClass, listField);
        //ConcurrentModificationException 발생을 거부합니다.
        Iterator<Field> iter = listField.iterator();
        while (iter.hasNext()) {
            Field field = iter.next();
            Caption annoCaption = field.getAnnotation(annotation);
            if (annoCaption == null)
                iter.remove();
        }
        return listField;
    }
}
```

TableDisplayer.java

```
public class TableDisplayer {
    public static String displayHeader(Class<?> targetClass) {
        StringBuilder sb = new StringBuilder("<tr>");
        List<Field> listField = ClassUtil.getListField(targetClass, Caption.class);
        for (Field field : listField) {
            Caption annoCaption = field.getAnnotation(Caption.class);
            if (annoCaption.isTableTarget(annoCaption.whenUse())) {
                sb.append("<th>").append(annoCaption.caption()).append("</th>");
            }
        }
        sb.append("</tr>");
        return sb.toString();
    }
}
```

01



데이터 안정성 강화

Iterator 활용

- 컬렉션 순회 안정성 확보
- 반복 처리 중 예외 발생 방지

02



시스템 성능 개선

StringBuilder 활용

- 불필요한 객체 생성 감소
- 메모리 사용량 최적화

■ 구 현 [ 3-8 ] 기술적 문제 해결



## 4

## DEBUG&amp;TEST

## [ 4-1 ] 오류 발견과 디버깅 - 유형 요약

| 오류 유형                    | 주요 원인                             |
|--------------------------|-----------------------------------|
| 1 AccessError 오류         | 권한 설정 오류 / Spring Security 설정 누락  |
| 2 Data Value 오류          | Controller → View 값 전달 오류, 매핑 불일치 |
| 3 Javascript Function 오류 | 이벤트 미작동, DOM 선택자 오류               |
| 4 AJAX 통신 오류             | 서버 응답 오류, 잘못된 URL/데이터 형식          |

## 4

## DEBUG &amp; TEST

## [ 4-2 ] 오류 발견과 디버깅 - 유형 별 디버깅

CustomAccessDeniedHandler.java security-context.xml

```

1 package www.dream.com.party.security;
2
3 import java.io.IOException;
4
11
12 public class CustomAccessDeniedHandler implements AccessDeniedHandler {
13
14     @Override
15     public void handle(HttpServletRequest request, HttpServletResponse response,
16         AccessDeniedException accessDeniedException) throws IOException, ServletException {
17         accessDeniedException.printStackTrace();
18         response.sendRedirect("/accessError");
19     }
20
21 }
22

```

| Name                                     | Value                                          |
|------------------------------------------|------------------------------------------------|
| no method return value                   |                                                |
| this                                     | CustomAccessDeniedHandler (id=110)             |
| request                                  | HttpServletRequestFactory\$Servlet3Secur...    |
| response                                 | HeaderWriterFilter\$HeaderWriterResponse (i... |
| accessDeniedException                    | AccessDeniedException (id=126)                 |
| backtrace                                | Object[6] (id=136)                             |
| cause                                    | AccessDeniedException (id=126)                 |
| depth                                    | 60                                             |
| detailMessage                            | "Access is denied" (id=138)                    |
| stackTrace                               | StackTraceElement[0] (id=144)                  |
| suppressedExceptions                     | Collections\$EmptyList<E> (id=145)             |
| <Choose a previously entered expression> |                                                |
| Access is denied                         |                                                |

오류 유형

디버깅 방법

1

AccessError 오류

AccessDeniedHandler 권한 확인 + accessDeniedException.printStackTrace(); + Breakpoint

## 4

## DEBUG &amp; TEST

## [ 4-3 ] 오류 발견과 디버깅 - 유형 별 디버깅

Withdraw Member

| Check                               | ID | Login ID    | Name    | Nickname  | BirthDate  | RegistrationDate |
|-------------------------------------|----|-------------|---------|-----------|------------|------------------|
| <input type="checkbox"/>            | 5  | no1pgmember | member  | pgmember1 | 1990-01-01 | 2021-02-24       |
| <input type="checkbox"/>            | 6  | no2pgmember | member  | pgmember2 | 1993-02-17 | 2021-02-24       |
| <input checked="" type="checkbox"/> | 41 | remove1     | remove1 | remove1   | 2021-03-12 | 2021-03-06       |
| <input checked="" type="checkbox"/> | 42 | remove2     | remove2 | remove2   | 2021-03-01 | 2021-03-06       |

Variables
Breakpoints
Expressions

| Name                   | Value                    |
|------------------------|--------------------------|
| no method return value |                          |
| this                   | PartyController (id=131) |
| arrTargetParty         | (id=132)                 |
| [0]                    | 41                       |
| [1]                    | 42                       |

<Choose a previously entered expression>

[41, 42]

```

193 @PostMapping("removeAdminParty")
194 public ResponseEntity<Integer> removeAdminParty(long[] arrTargetParty){
195     System.out.println(arrTargetParty);
196     return new ResponseEntity<>(partyService.removeAdminParty(arrTargetParty), HttpStatus.OK);
197 }

```

오류 유형

디버깅 방법

2

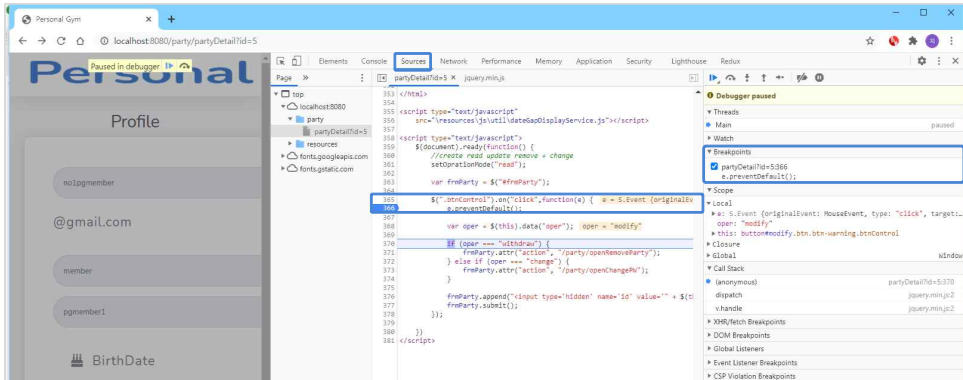
Data Value 오류

컨트롤러 매핑 부분에 Breakpoint → Variable 탭 확인 또는 sysout(매핑한 field명) 선언 Breakpoint

## 4

## DEBUG &amp; TEST

## [ 4-4 ] 오류 발견과 디버깅 - 유형 별 디버깅



오류 유형

디버깅 방법

3

Javascript Function 오류

개발자도구(F12) → Sources 탭 → Function 일 경우 `e.preventDefault()`; 줄번호 클릭, Breakpoint

localhost:8080 내용:

code:500

```
message:<!doctype html><html lang="ko"><head><title>HTTP
상태 500 - 내부 서버 오류</title><style type="text/css">body
{font-family:Tahoma,Arial,sans-serif;} h1, h2, h3, b
{color:white;background-color:#525D76;} h1 {font-size:22px;} h2
{font-size:16px;} h3 {font-size:14px;} p {font-size:12px;} a
{color:black;} .line {height:1px;background-
color:#525D76;border:none;}</style></head><body><h1>HTTP 상
태 500 - 내부 서버 오류</h1><hr class="line" /><p><b>타입</
b> 예외 보고</p><p><b>메시지</b> Request processing failed;
nested exception is org.mybatis.spring.MyBatisSystemException:
nested exception is org.apache.ibatis.builder.BuilderException: The
expression &#39;array&#39; evaluated to a null value.</
p><p><b>설명</b> 서버가, 해당 요청을 충족시키지 못하게 하는
예기치 않은 조건을 맞닥뜨렸습니다.</p><p><b>예외</b></
p></pre>
```

```
p><pre>org.springframework.web.util.NestedServletException:
Request processing failed; nested exception is
org.mybatis.spring.MyBatisSystemException: nested exception is
org.apache.ibatis.builder.BuilderException: The expression
&#39;array&#39; evaluated to a null value.
~org.springframework.web.servlet.FrameworkServlet.processRequest
t(HttpServletRequest
t.java:1014)
~org.springframework.web.servlet.FrameworkServlet.doPost(Framew
orkServlet.java:9
09)
~javax.servlet.http.HttpServlet.service(HttpServlet.java:652)
~org.springframework.web.servlet.FrameworkServlet.service(Framew
orkServlet.java:
883)
~javax.servlet.http.HttpServlet.service(HttpServlet.java:733)
...
error:function 핸들러 코드
```

error:function 핸들러 코드

```
error:function(request,status,error){ alert("code:"+request.status
+"Wn"+"message:"+request.responseText+"Wn"+"error:"+error); }
```

오류 유형

디버깅 방법

## 테스트 목적

- Party, Post, Reply, HashTag 조회 기능 검증
- 댓글·대댓글 계층 구조 및 데이터 연관 관계 확인
- 주요 Mapper 조회 쿼리의 정상 동작 확인

## 검증 방식

- 조회 결과 검증
- 데이터 연관 관계 확인

## 테스트 항목

|   |         |             |
|---|---------|-------------|
| 1 | Party   | 회원 조회       |
| 2 | Post    | 게시글 조회      |
| 3 | Reply   | 댓글 / 대댓글 조회 |
| 4 | HashTag | 해시태그 조회     |



```
@RunWith(SpringJUnit4ClassRunner.class)
@ContextConfiguration("file:src\\main\\webapp\\WEB-INF\\spring\\root-context.xml")
public class PartyMapperTest {
```

```
    @Autowired
    private PartyMapper partyMapper;
```

```
    @Test
    public void testPerson() {
        try {
            assertNotNull(partyMapper);
            for (PersonVO person : partyMapper.selectAllPerson())
                System.out.println(person);
        } catch (Exception e) {
            e.printStackTrace();
        }
    }
```

```
    @Test
    public void testPersonWithCP() {
        try {
            assertNotNull(partyMapper);
            for (PersonVO person : partyMapper.selectAllPersonWithContactPoint())
                System.out.println(person);
        } catch (Exception e) {
            e.printStackTrace();
        }
    }
}
```

### Test Person

#### 1 회원 조회 기능 검증

- Mapper Bean 정상 주입 확인  
- assertNotNull 이용 → partyMapper null 여부 확인
- 전체 회원 조회 결과 검증  
- Foreach문 이용 → 모든 멤버(selectAllPerson)의 회원정보 조회 → PersonVO의 레코드 출력
- 조회된 회원 정보를 로그로 출력, 데이터 정상 확인

```
Problems | Javadoc | Declaration | Console | Servers
terminated- PartyMapperTest [JUnit] C:\Users\Wuer-004\Downloads\wsts-4.8.1.RELEASE\plugins\org.eclipse.justi.openjdk.hotspot.jre.full.win32.x86_64_15.0.0.v20201014-1246\jre\bin\java.exe (2020. 12. 4. 5)
SLF4J: Failed to load class "org.slf4j.impl.StaticLoggerBinder".
SLF4J: Defaulting to no-operation (NOP) logger implementation
SLF4J: See http://www.slf4j.org/codes.html#StaticLoggerBinder for further details.
PersonVO [id=1, name=장연자, birthDate=Thu Jan 01 00:00:00 KST 1920, regDate=Fri Dec 04 1:
PersonVO [id=1, name=장연자, birthDate=Thu Jan 01 00:00:00 KST 1920, regDate=Fri Dec 04 1:
```

```

@RunWith(SpringJUnit4ClassRunner.class)
@ContextConfiguration("file:src\\main\\webapp\\WEB-INF\\spring\\root-context.xml")
public class ReplyMapperTest {
    @Autowired
    private ReplyMapper replyMapper;

    @Test
    public void testFindAllPost() {
        try {
            for (ReplyVO reply : replyMapper.findAllPost(1L))
                System.out.println(reply);
        } catch (Exception e) {
            e.printStackTrace();
        }
    }
}

```

## Test Post

## 2 게시물 조회 기능 검증

- 특정 작성자(writer\_id)의 게시물 목록 조회
  - 조회 조건 : id 값이 일치하는 writer
- 조회 조건에 맞는 게시물 반환 여부 확인
  - Foreach문 이용 → 특정 작성자가 작성한 게시물 (findAllPost) 조회 → PostVO의 레코드 출력
- 게시물 조회 기능 정상 동작 검증

Problems @ Javadoc Declaration Console Servers  
 terminated- ReplyMapperTest [JUnit] C:\Users\user-004\Downloads\sts-4.6.1.RELEASE\plugins\org.eclipse.just.openjdk.hotspot.jre.full.win32.x86\_64\_15.0.0.v20201014-1240\jre\bin\java.exe (2020.12.8 오전 10:23:07 - 오전 10:23:12)  
 PostVO [id=1, title=무궁한 발전 기원, content=많은 사람들에게 도움을..., writer=PersonVO [id=1, name=창업자, birthDate=null, regDa^  
 PostVO [id=2, title=직원 모집 공고, content=열의에 가득찬 사람들 오세요, writer=PersonVO [id=1, name=창업자, birthDate=null, regDat

## 4

## DEBUG &amp; TEST

[ 4-9 ] JUnit Test - Reply

```

@Test
public void testFindAllReply() {
    try {
        for (ReplyVO reply : replyMapper.findAllReply(2L))
            System.out.println(reply);
    } catch (Exception e) {
        e.printStackTrace();
    }
}

```

## Test Reply/Re-Reply

## 3 댓글 및 대댓글 조회 기능 검증

- 특정 게시글의 댓글(Reply) 목록 조회

- Foreach문 이용 → 특정 게시글에 달린 댓글  
(findAllReply) 조회 → ReplyVO의 레코드 출력

- 특정 댓글의 대댓글(Re-Reply) 목록 조회

- Foreach문 이용 → 특정 댓글에 달린 대댓글  
(findReplyWithReply) 조회 → ReplyVO의 레코드 출력

- 계층형 댓글 구조 정상 동작 확인

```

@Test
public void testFindReplyWithReply() {
    try {
        for (ReplyVO reply : replyMapper.findReplyWithReply(3L))
            System.out.println(reply);
    } catch (Exception e) {
        e.printStackTrace();
    }
}

```

istReplies=[ReplyVO [id=5, content=findReplyWithReply test 대댓글, writer=PersonVO [id=1, name=장업자, birthDate=null, regDate=null, updateDate=

## 4

## DEBUG &amp; TEST

[ 4-10 ] JUnit Test - HashTag

```

@RunWith(SpringJUnit4ClassRunner.class)
@ContextConfiguration("file:src\\main\\webapp\\WEB-INF\\spring\\root-context.xml")
public class HashtagMapperTest {
    @Autowired
    private HashTagMapper hashTagMapper;

    @Test
    public void findExisting() {
        try {
            for (HashTagVO hashTagVO : hashTagMapper.findExisting(new String[] { "IT", "자두", "영등포" }))
                System.out.println(hashTagVO);
        } catch (Exception e) {
            e.printStackTrace();
        }
    }
}

```

## Test HashTag

## 4 해시태그 검색 기능 검증

- 입력된 키워드가 해시태그 DB에 존재하는지 확인
- 다중 키워드 검색 결과 검증
  - Foreach문 이용 → 존재하는 해시태그 문자열 조회 (findExisting) → HashTagVO의 레코드 출력
- 해시태그 조회 기능 정상 동작 확인

Problems Javadoc Declaration Console Servers  
 <terminated> HashtagMapperTest [Unit] C:\Users\user-004\Downloads\sts-4.8.1.RELEASE\plugins\org.eclipse.justj.openjdk.hotspot.jre.full.win32.x86\_64\_15.0.0.v20201014-1246\jre\bin\javaw.exe (2020. 12. 14 오후 5:48:18 - 오후 5:48:24)

SLF4J: See <http://www.slf4j.org/codes.html#StaticLoggerBinder> for further details.

HashTagVO(id=1, word=IT, describe=banana, superHashTag=null, childHashTag=null)

## Strength

- CRUD 전체 기능 구현
- Spring MVC 경험
- Spring Security 권한 관리 구현

## Weakness

- 실제 서버 배포 미적용
- 테스트 자동화 부족
- 대규모 트래픽 검증 부재

## Opportunity

- 운동 기록 기능 추가
- 운동 코칭 서비스 연계
- 모바일 서비스 확장

## Threat

- 기술 변화 대응
- 보안 리스크 관리
- 장애 대응 체계 구축

## 5

## PROJECT REVIEW

## [ 5-2 ] 프로젝트 성과

## 기술적 성과



## CRUD 전체 구현

- 게시글, 댓글, 파일 등 주요 기능 구현
- 안정적인 데이터 처리



## 권한 관리 구현

- Spring Security 적용
- 회원/비회원/관리자 권한 분리



## 검색 기능 구현

- 해시태그 기반 검색
- 형태소 분석(Komoran) → 검색 정확도 향상



## 계층형 댓글 구조

- 댓글/대댓글 구현
- obj\_type 기반 계층 구조 설계

→ 안정적이고 확장 가능한 웹서비스를 구현

## 학습 및 성장 성과



## 설계 역량 향상

- UML 기반 설계 경험



## 데이터베이스 역량 강화

- DB 설계 및 SQL 최적화 경험



## 협업 능력 향상

- 공동 개발 및 코드 리뷰 경험



## 문제 해결 능력 향상

- 디버깅, 테스트를 통한 품질 향상

→ 개발자로서의 역량을 한 단계 성장시킨 프로젝트



1

서비스 운영 환경 구축

- AWS 기반 서버 배포
- Docker 컨테이너 적용
- GitHub Actions 기반 CI/CD 구축

2

사용자 경험 개선

- 모바일 어플리케이션 서비스 확장
- 다양한 디바이스 환경 대응

3

아키텍처 고도화

- Spring Boot 기반 구조 전환
- 로그 및 모니터링 체계 구축

## 6

## REFERENCE

## [ 6-1 ] 팀구성 및 협업방식

## 팀 구성



2인 참여 프로젝트  
역할 구분 없이 협업



## Pair Programming

설계, 구현, 테스트를  
공동으로 수행

## 협업 방식



## 커뮤니케이션

데일리 실시간 미팅으로  
진행 상황 공유 및 이슈논의



## 코드 리뷰

상호 코드 리뷰 및  
피드백을 통해 품질 향상

## 개발 환경 및 도구



개발 환경  
Eclipse, Oracle,  
Spring, MyBatis



버전 관리  
GitHub



소통 도구  
카카오톡, 구글 드라이브



- 프로젝트 참고사항

- 학습 및 포트폴리오 목적의 팀 프로젝트로 실제 서버 배포 없이 개발을 완료

- github

<https://github.com/ztoacej-personal/PersonalGYM.git>

- 프로젝트 문의사항

- 프로젝트 관리자 : 차은지 ( atozcej@gmail.com )

- 포트폴리오 참고사항

- 01\_PersonalGYM\_Portfolio (개발 과정 중심 ppt)

- 02\_PersonalGYM\_Service Demo (서비스 기능 중심 ppt)